

Bakir, Abdil Karim

Software Engineering · Backend, Web, Cloud, Hardware



LICHTBILD

VERFÜGBAR	Ab sofort · Werkstudent in Teilzeit
STANDORT	Berlin, Deutschland
E-MAIL	abdilkarimb@gmail.com
GITHUB	github.com/KarimBk7
ONLINEFASSUNG	my-website.abdilkarimb.workers.dev
STAND	26. September 2026

34 mal schnellere Prüfung
bei 10.000 Zeilen, Spesenkonfigurator

67/67 abgenommene Aufgaben
Programmierpraktikum der FU Berlin

145 ms Antwortzeit
dieser Seite, Median aus fünf Abrufen

INHALT

M-1	Spesenkonfigurator für Projektron BCS · Übergeben 07/2026
M-2	ReflowTask · In Betrieb 09/2026
M-3	Diese Seite · Veröffentlicht 09/2026
M-4	Kaiju Adventure · Abgegeben 12/2022
M-5	Technisches Lern-Repository · Abgenommen 67/67
M-6	Wordle-Solver · Veröffentlicht v1.0

Dieses Dokument wird aus der Bewerbungsseite erzeugt und trägt denselben Stand. Videos und die interaktive Demo gibt es nur online; im Dokument steht an ihrer Stelle ein Standbild. Alle verlinkten Adressen sind ausgeschrieben.

Spesenkonfigurator für Projektron BCS

Windows-Desktop-Anwendung, mit der Personal- und Buchhaltungsteams die CSV-basierte Reisekosten-Konfiguration von Projektron BCS pflegen, ohne das zugrunde liegende Dateiformat zu kennen.

AUFTRAGGEBER	Projektron GmbH
RAHMEN	Softwareprojekt, FU Berlin
ZEITRAUM	13.04. – 01.08.2026
TEAM	6 Personen
MEINE ROLLE	Backend Architect

AUFGABE

Die Konfiguration legt für jede Kombination aus Arbeits- und Reiseland fest, welche Pauschalen, Schwellenwerte und Kürzungsregeln in welchem Zeitraum gelten. Von Hand gepflegt war sie unübersichtlich und fehleranfällig, zumal Excel beim Öffnen Geldbeträge in Datumsangaben umwandelt und keine fachliche Prüfung bietet.

Die technische Kernfrage

JavaFX kennt kein Anheften einzelner Spalten. Die fixierte linke Spalte ist deshalb eine zweite Tabelle, die über den Pixel-Offset der virtualisierten Darstellung synchron gehalten wird. Der Stresstest mit 10.000 Zeilen zeigte danach, dass nicht die Prüflogik langsam war, sondern deren Rückmeldung an die Tabelle: ein Listen-Beobachter löste rund 9.600 einzelne Oberflächen-Ereignisse aus. Ihn durch ein einziges Signal je Prüfdurchlauf zu ersetzen brachte den Faktor 34.

MEIN BEITRAG

Tabellenkern aus zwei pixelgenau synchronisierten Tabellen mit fixierter Spalte

Multi-Tab-Verwaltung mit vollständig gekapseltem Zustand je Datei

Undo/Redo nach dem Memento-Muster mit dateigrößenabhängigem Speicherbudget

Mehrsprachigkeit mit Sprachwechsel zur Laufzeit

Absturzsicherung mit automatischer Wiederherstellung des Bearbeitungsstands

Performance-Optimierung nach dem Stresstest mit 10.000 Zeilen

368

eigene Commits
von 703 im Repository

3

behobene Speicherlecks
im Code-Review aufgedeckt

19

Testklassen
plus Prüftreiber für Last und Oberfläche

90+

Klassen
im übergebenen Quelltext

PERFORMANCE-OPTIMIERUNG

Ein großer Teil meiner Arbeit ging in die Performance. Nach dem Stresstest mit 10.000 Zeilen habe ich jede Optimierung einzeln vorher und nachher gemessen; die vier wichtigsten Werte stehen hier. Sie stammen aus der Projektdokumentation, nicht aus der Erinnerung.

PRÜFPUNKT	VORHER	NACHHER
Prüfung nach jeder Änderung 10.000 Zeilen	5.051 ms	148 ms Faktor 34×
Datei laden und darstellen 10.000 Zeilen	5.000 ms	600 ms Faktor ≈8×
Prüfbericht bei 10.000 Fehlern 10.000 Zeilen	MEHRERE MINUTEN	283 ms
Oberflächen-Ereignisse je Prüfdurchlauf 10.000 Zeilen	9.600	1 Faktor 9600 : 1

Gemessen mit einem TestFX-Treiber, der die echte Oberfläche startet und 10.000 Zeilen in die reale Tabelle einspeist – kein synthetischer Aufbau. Reproduzierbar über `mvn test -Dtest=PerfVerifyDriver`.

DER EINGRIFF

Vier Zeilen weniger, ein Signal mehr. Bevor das feststand, habe ich Schicht für Schicht gemessen, wo die Zeit blieb — die Prüflogik war es nicht.

nur die Liste		142 ms
+ Extractor		250 ms
+ Filtered/Sorted		260 ms
+ TableView		5.051 ms

95 % der Kosten lagen darin, dass die Tabelle auf rund 9.600 einzelne Ereignisse reagierte – nicht im Prüfen selbst.

VORHER

```
// Extractor tells JavaFX to fire list-change
// events when hasRowError/hasRowWarning change
private final ObservableList<ExpenseEntry> data =
    FXCollections.observableArrayList(
        entry -> new Observable[] {
            entry.hasRowError,
            entry.hasRowWarning});
```

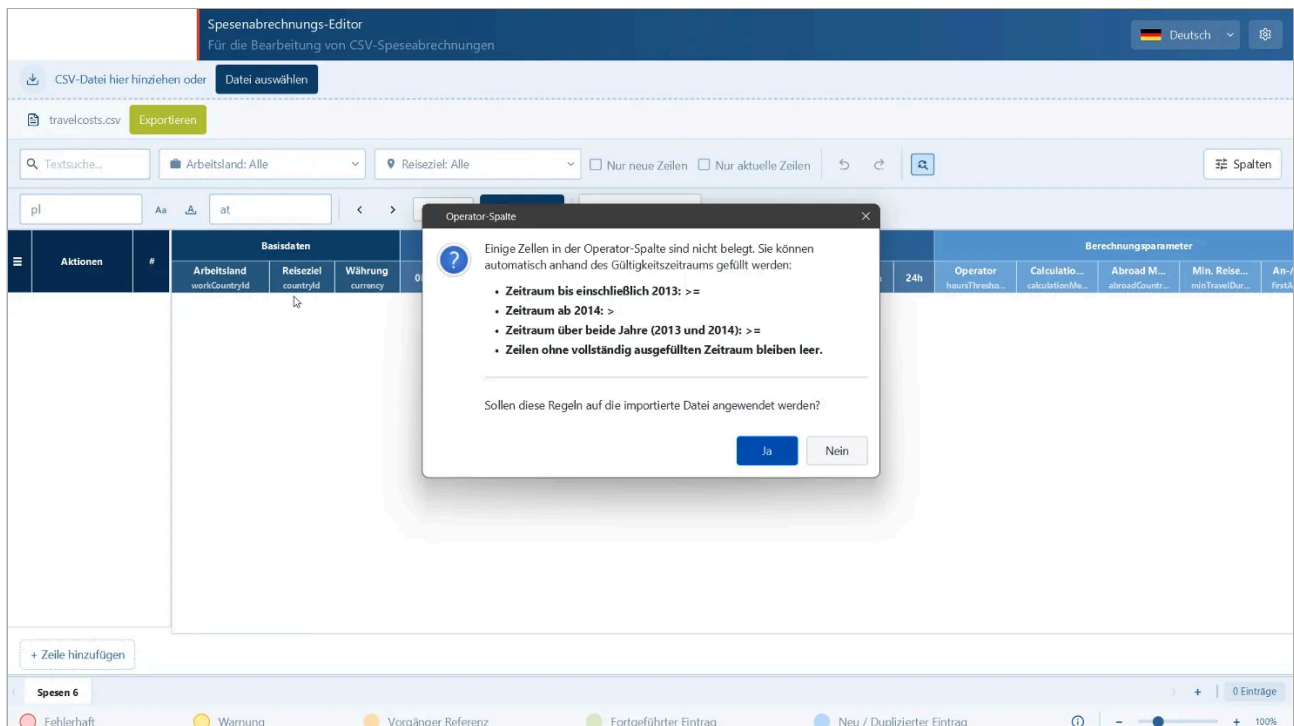
NACHHER

```
private final ObservableList<ExpenseEntry> data =
    FXCollections.observableArrayList();

// Bumped once after each full validation pass
private final IntegerProperty validationEpoch =
    new SimpleIntegerProperty(0);

public void markValidated() {
    validationEpoch.set(validationEpoch.get() + 1);
}
```

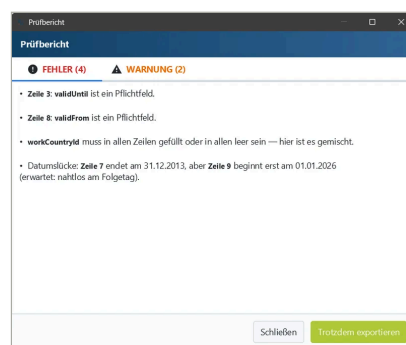
ExpenseManager.java · Commit f4e8223



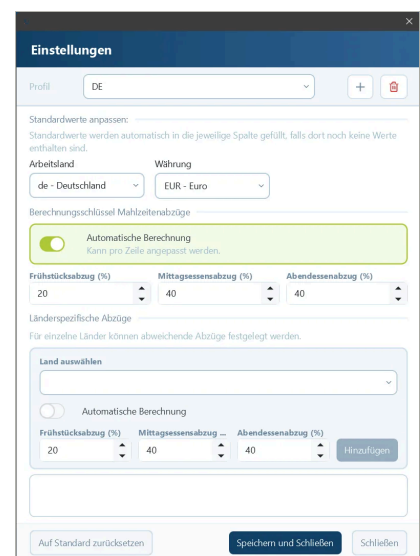
Laufende Bedienung, 24 Sekunden: Nach dem Import schlägt ein Dialog vor, leere Operator-Zellen anhand des Gültigkeitszeitraums zu füllen. Danach Filtern nach Reiseziel, Suchen und Ersetzen, eine Zelle bearbeiten, eine gelb markierte Warnung, Scrollen durch die Spaltengruppen, die Einstellungen und das Speichern.



Zeilenassistent: geführte Erfassung Feld für Feld, mit länderspezifischen Schritten.



Prüfbericht: sammelt alle Befunde und springt per Klick zur Zelle.



Einstellungen: Vorgabewerte je Land, speicherbar als benannte Profile.

Was ich heute anders bauen würde

Drei Stellen aus meinem eigenen Entscheidungsprotokoll. Sie stehen dort seit der Projektzeit als bewusst eingegangene Kompromisse — nicht im Nachhinein zurechtgelegt.

Die fixierte Spalte hängt an `fixedCellSize`. Das ist schnell, aber empfindlich: nach Struktur- und Spaltenänderungen bleiben Zellen unpositioniert, bis ein Ein-Pixel-Scroll die Ansicht anstößt. Ein bewusster Behelf, kein Entwurf. (D3)

Undo kopiert bei jeder Aktion die gesamte Zeilenliste, rund 2,7 KB je Zeile. Ein delta-basiertes Undo hatte ich geprüft und als zu aufwendig verworfen — die Speichergrenze musste ich später doch nachziehen. (D5, korrigiert in D22)

Die Dialog-Helfer in `util` importieren ihre Controller direkt. Sauber wäre je eine Schnittstelle; ich habe das für FXML-Dialoge als Überbau eingestuft. (D10)

WERKZEUGE

Java 21	JavaFX 21	FXML	Maven	JUnit 5
TestFX	CSS	jpackage	GitHub Actions	Spotless

AUSLIEFERUNG

Übergeben als nativer Windows-Installer mit gebündelter Java-Laufzeitumgebung, installierbar ohne separate Java-Installation und ohne Administratorrechte. Mitgeliefert wurden Entscheidungsprotokoll, Validierungsregel-Katalog und acht How-to-Anleitungen.

EINORDNUNG

Die genannten Funktionen stammen aus meinem Verantwortungsbereich im sechsköpfigen Team. Oberfläche, Prototyping und Anforderungsmanagement lagen bei den übrigen Teammitgliedern, die fachliche Validierungslogik entstand gemeinsam. Der Quelltext gehört dem Auftraggeber und ist deshalb als einziges der hier gezeigten Projekte nicht öffentlich einsehbar.

ReflowTask

Aufgabenplaner, der sich selbst repariert: Jede Aufgabe bekommt echte Zeitblöcke, und wer einen verpasst, bekommt den Rest automatisch neu geplant. Selbst gehostet für einen Haushalt, im Betrieb auf einem Raspberry Pi 5.

RAHMEN	Eigenarbeit
ZEITRAUM	14.09. – 24.09.2026
MEINE ROLLE	Backend, Oberfläche, Betrieb
UMFANG	3.517 Zeilen Java, 4.300 Zeilen TypeScript
BETRIEB	Raspberry Pi 5, Docker Compose

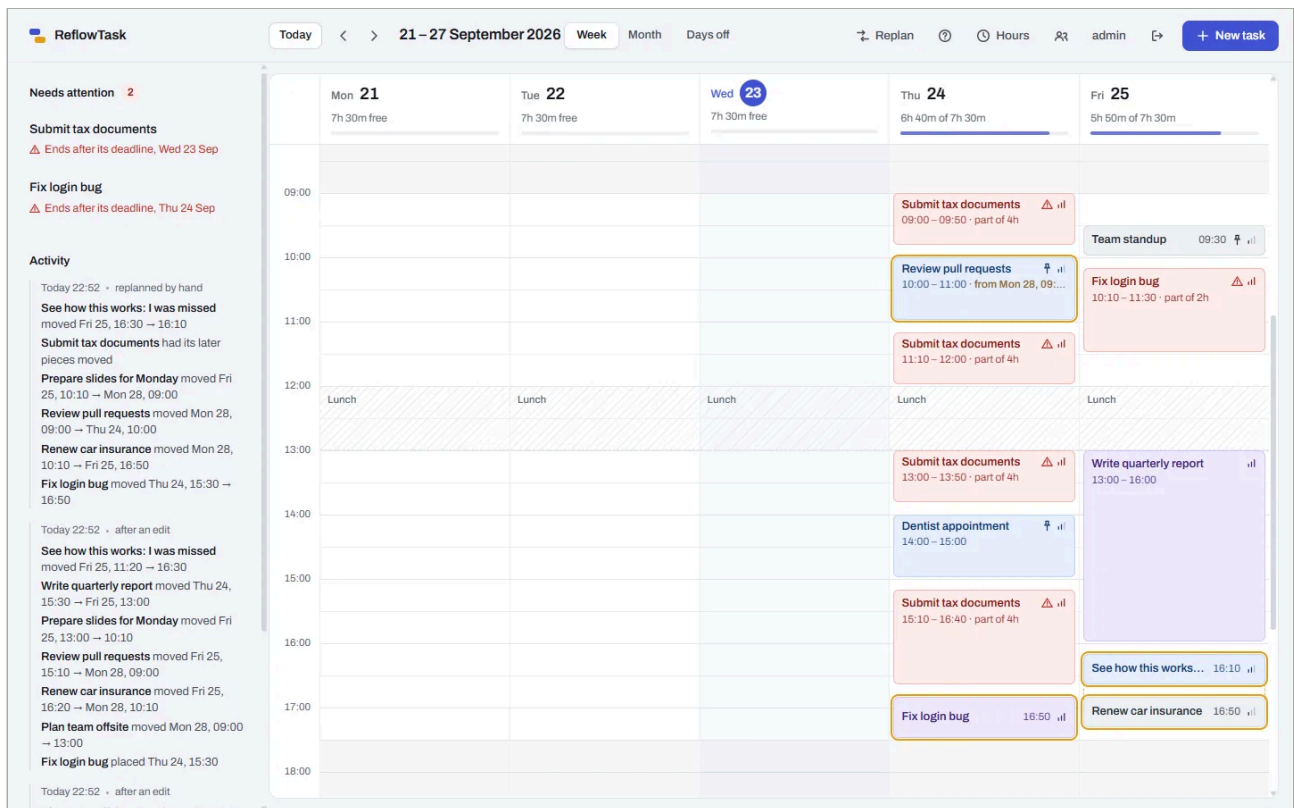
AUFGABE
Aufgabenlisten sagen, was zu tun ist, aber nicht wann. Wird ein Termin verpasst, steht die Aufgabe rot da und das Umräumen des Tages bleibt am Menschen hängen. Fertige Dienste, die selbst umplanen, kosten ein Abo und legen den Kalender auf fremde Server. ReflowTask macht dasselbe auf eigener Hardware: Jede Aufgabe trägt eine geschätzte Dauer und optional eine Frist, daraus werden Blöcke in den eigenen Arbeitszeiten.

Planen als reine Funktion

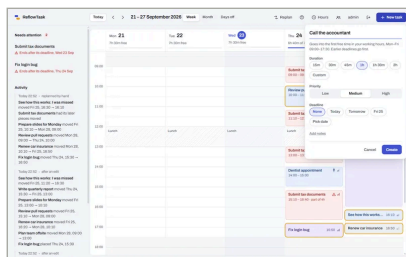
Der Kern ist bewusst eine reine Funktion: Aufgaben, belegte Zeiten, Einstellungen und der aktuelle Zeitpunkt hinein, fertige Blöcke heraus — ohne Datenbank, ohne Uhr von außen. Dadurch ist jede Regel einzeln durch einen Test beweisbar. Die Reihenfolge ist Frist zuerst, dann Priorität, dann das Alter der Aufgabe. Der letzte Schlüssel ist kein Schönheitsfehler, sondern Absicht: Ohne ihn würden gleichwertige Aufgaben bei jedem Lauf die Plätze tauschen, und jede Umplanung sähe nach einer Änderung aus. Freie Zeit beginnt außerdem immer auf einer glatten Viertelstunde, sonst steht nach einer Umplanung um 10:37 Uhr ein Kalender voller 10:37-Termine.

MEIN BEITRAG
Planungskern als reine Funktion, direkt testbar ohne Datenbank
Stündlicher Lauf erkennt verpasste Blöcke und plant nur den Rest neu
Zustände je Block (geplant, erledigt, verpasst): erledigte Teile zählen gegen die Schätzung
Haushaltskonten mit eigenen Aufgaben, Arbeitszeiten und Verlauf je Person
Datentrennung gegnerisch geprüft: fremde IDs antworten mit 404, nicht mit 403
Anmeldung mit Sitzungscookies, bcrypt und Sperre nach fünf Fehlversuchen
Auslieferung als Docker Compose, im Betrieb auf einem Raspberry Pi 5 hinter Caddy und Tailscale

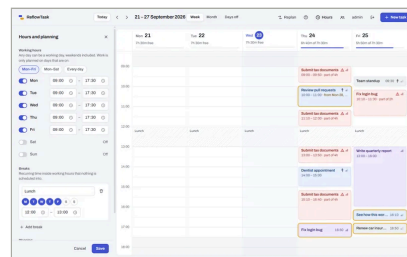
KENNWERTE			
168 Backend-Tests, alle grün am 24.09.2026 ausgeführt, dazu 29 im Frontend	7 Datenbank-Migrationen laufen gleich auf H2 und PostgreSQL	40 Commits in zehn Tagen, 14.–24.09.2026	59 Java-Klassen plus 26 Dateien im Frontend



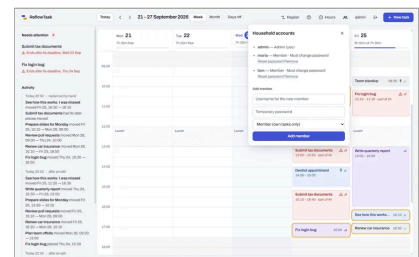
Wochenansicht: geplante Blöcke in den Arbeitszeiten, rot markiert, was nach seiner Frist endet, gelb umrandet ein verschobener Block samt Herkunft. Links die Aufgaben, die Aufmerksamkeit brauchen, und darunter das Protokoll jeder Verschiebung. Oben je Tag, wie viel Zeit noch frei ist.



Neue Aufgabe: Dauer, Priorität und Frist über Schaltflächen statt über ein Formular. Der Satz darüber sagt vorab, wohin die Aufgabe fällt.



Arbeitszeiten je Person: Vorlagen für Mo–Fr, Mo–Sa oder jeden Tag, dazu Pausen, in die nichts geplant wird. Schichtwoche statt fest verdrahtetem Bürotag.



Haushaltskonten: Mitglieder anlegen, Passwörter zurücksetzen, Konten entfernen. Jede Person sieht nur ihre eigenen Aufgaben; Rolle und Passwortpflicht stehen an der Zeile.

Was noch fehlt

Im Betrieb, aber nicht fertig. Diese Punkte sind bekannt und stehen hier, damit niemand sie erst suchen muss.

Keine wiederkehrenden Aufgaben

Kein Abgleich mit fremden Kalendern: Ein Termin von außen blockiert nur, wenn er auch hier steht

Die Anmeldesperre liegt im Arbeitsspeicher und ist nach einem Neustart weg

Die Oberfläche gibt es nur auf Englisch, alle Texte stehen aber in einer Datei

Keine Erinnerungen aufs Telefon; der vorgesehene Weg über einen Kalender-Feed fehlt noch

WERKZEUGE

Java 25	Spring Boot 4.1	Spring Data JPA		
PostgreSQL	Flyway	React 19	TypeScript	Vite
TanStack Query	JUnit 5	Vitest	Docker Compose	
GitHub Actions				

AUSLIEFERUNG

Ein Befehl mit Docker Compose bringt PostgreSQL, das Spring-Boot-Backend und die über Nginx ausgelieferte Oberfläche hoch. Läuft im Betrieb auf einem Raspberry Pi 5 hinter Caddy, erreichbar über Tailscale statt über das offene Internet. Eine Neuinstallation von Grund auf wurde durchgespielt, um die Anleitung zu prüfen.

EINORDNUNG

Private Arbeit, Quelltext und Oberfläche stammen vollständig von mir. Bewusst ohne vollständiges Spring Security: Das Bedrohungsmodell ist ein Haushalt im privaten Netz, und die Begründung dafür steht als eigenes Dokument im Repository. Passwörter setzt zurück, wer Zugang zur Maschine hat — es gibt keinen Mailserver, also auch keinen Link zum Zurücksetzen.

github.com/KarimBk7/ReflowTask

(<https://github.com/KarimBk7/ReflowTask>)

Diese Seite

Die Bewerbungsseite, die Sie gerade lesen: statisch erzeugt, zweisprachig, ohne Tracker und ohne Cookies, ausgeliefert als Cloudflare Worker. Ein Skript von 5,2 KB je Seite, kein Framework im Browser; auf dem Blatt M-6 kommt der Solver als zweites Skript dazu.

RAHMEN	Eigenarbeit
ZEITRAUM	September 2026
MEINE ROLLE	Entwurf, Umsetzung, Betrieb
BETRIEB	Cloudflare Worker
UMFANG	Zwei Sprachen, eine Quelle

AUFGABE Eine Bewerbungsseite, die das belegt, was sie behauptet: schnell, ohne fremde Dienste, ohne Datensammlung, mit Inhalten aus einer einzigen geprüften Quelldatei statt handgepflegtem HTML. Bilder werden beim Bauen einmal in mehreren Größen erzeugt, nicht zur Laufzeit umgerechnet. Was lokal grün ist, ist online noch lange nicht grün Drei Fehler haben es trotz vollständig grüner lokaler Prüfung bis auf den Server geschafft: die Sicherheitsrichtlinie verbot eingebettete Skripte, weshalb das Seitenverhalten stumm ausfiel; die Bilder wurden zur Laufzeit angefordert und liefen ins Leere; und eine zu lange Cache-Frist lieferte neues HTML mit altem JavaScript aus. Alle drei schlugen nirgends fehl — sie taten einfach nichts. Daraus wurde ein zweiter Prüflauf, der nicht die gebaute Seite testet, sondern die veröffentlichte: ob die Kopfzeilen wirklich ankommen, ob jedes Bild lädt und ob das Skript tatsächlich gelaufen ist.	MEIN BEITRAG Inhalte in einer einzigen typisierten Quelldatei, beide Sprachen daneben statt in getrennten Dateien Sicherheitsrichtlinie ohne eingebettete Skripte, dazu nosniff, Referrer-Regel und gesperrte Einbettung Eigene Bild-Pipeline: WebP in zwei Breiten beim Bauen, ohne je hochzurechnen Prüflauf gegen die veröffentlichte Adresse, nicht nur gegen den lokalen Bau Prüfung auf Querüberlauf bei 1440 und 390 Pixeln, auch während der Einblendbewegung Bewegung vollständig abschaltbar über prefers-reduced-motion
---	---

KENNWERTE			
145 ms Antwortzeit Median aus fünf Abrufen	5,2 KB JavaScript je Seite eine Datei, gzip, kein Framework	0 Tracker und Cookies keine fremden Server	2 Sprachen vollständig, aus einer Quelle

WERKZEUGE Astro TypeScript CSS Cloudflare Workers sharp Playwright GitHub Actions	EINORDNUNG Entwurf, Inhalte und Abnahme liegen bei mir; die Umsetzung ist im Dialog mit einem KI-Agenten entstanden. Das Repository ist offen — Quelltext, Prüfskripte und die Historie samt der oben genannten Fehler sind einsehbar.
--	--

github.com/KarimBk7/My-Website
(<https://github.com/KarimBk7/My-Website>)

Kaiju Adventure

Vollständiges 2D-Action-Adventure in Java, bewusst ohne Spiel-Engine gebaut: eigene Spielschleife, eigene Kollisionserkennung, kachelbasierte Welt und Spielstände als Datei, wahlweise in einer MySQL-Datenbank.

RAHMEN Abiturprojekt

ZEITRAUM Nov. – Dez. 2022

MEINE ROLLE Gesamter Quelltext

TEAM Grafik und Klänge

UMFANG 29 Klassen

AUFGABE

Ziel war ein fertig spielbares Adventure ohne Rückgriff auf eine Spiel-Engine, also mit selbst geschriebener Spielschleife, eigener Darstellung und eigener Kollisionsprüfung.

Die Spielschleife

Ein eigener Thread hält das Spiel auf 60 Bilder je Sekunde. Statt fester Wartezeiten sammelt ein Delta-Zeit-Akkumulator die vergangene Zeit und zeichnet erst, wenn ein volles Bildintervall zusammengekommen ist. Aktualisieren und Zeichnen sind getrennt. Auf schnelleren Rechnern läuft das Spiel dadurch nicht schneller.

MEIN BEITRAG

Gesamter Quelltext vom Rendering bis zur Datenbankanbindung

Spielschleife mit Delta-Zeit für konstante Bildrate

Kollisionserkennung gegen Kacheln, Objekte und Einheiten

Zustandsautomat über elf Spielzustände

Spielstände über JDBC in MySQL, später auf lokale Dateien als Standard umgestellt

Auslieferung als Windows-Programm mit eigener Java-Laufzeit

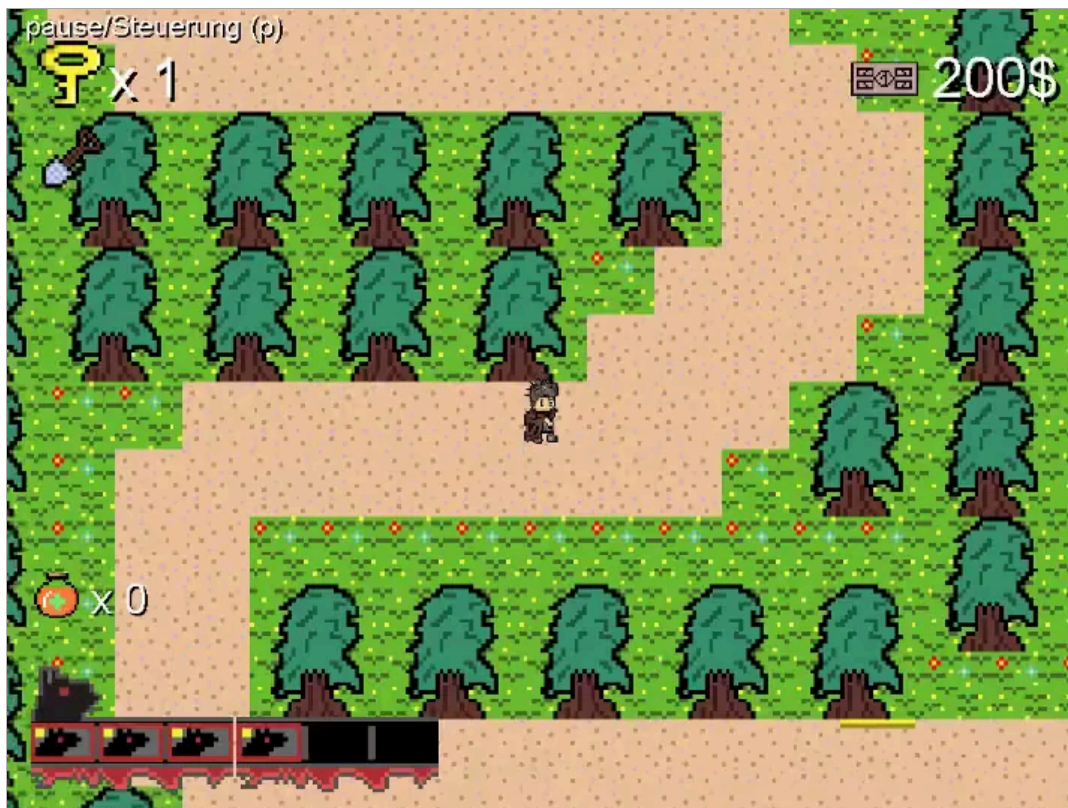
KENNWERTE

29 Java-Klassen
rund 4.960 Zeilen

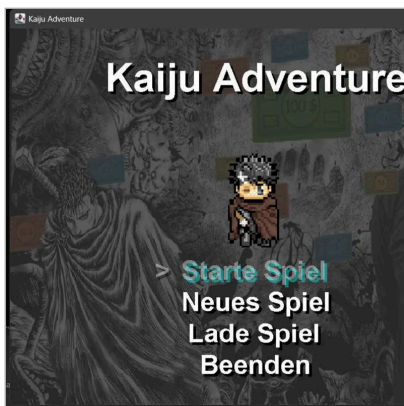
60 Bilder je Sekunde
eigener Game-Loop

6.400 Kacheln Weltkarte
80 × 80 zu je 48 px

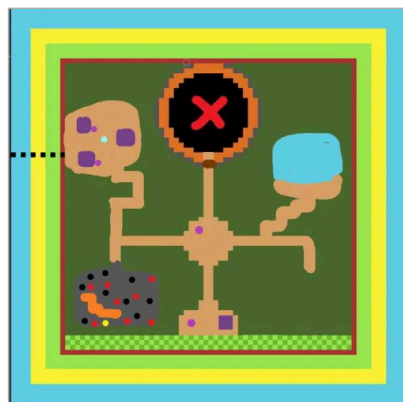
11 Spielzustände
von Titel bis Epilog



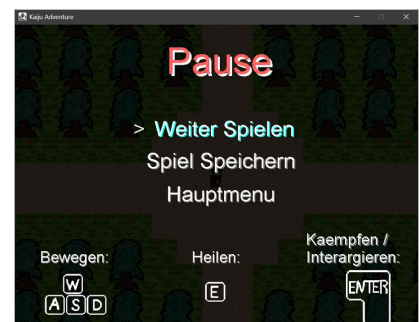
Laufende Spielszene, 20 Sekunden mit 60 Bildern je Sekunde: Die Kamera folgt der Spielfigur durch den Wald, durch das Tor ins ummauerte Dorf und bis zum Haus eines Dorfbewohners. Unterwegs ein Schwerthieb, gegnerische Würmer und die Geldanzeige, die im Lauf von 200 auf 300 springt.



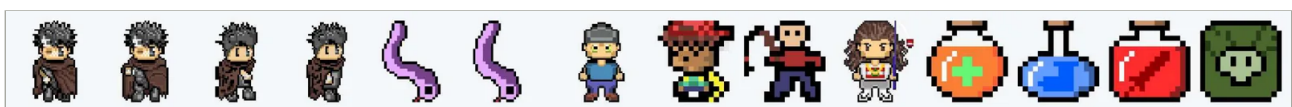
Titelbildschirm mit Menüführung über die Zustände Starten, Neues Spiel, Laden und Beenden. Die Auswahl läuft vollständig über den selbst geschriebenen Zustandsautomaten.



Entwurf der Welt mit Dorf, See, Höhle und Bossarena. Das Raster liegt als Textdatei im Projekt und wird beim Start in Kacheln übersetzt.



Pausemenü über der abgedunkelten Spielwelt — einer der elf Spielzustände. Weiterspielen, Speichern und Hauptmenü stehen zur Wahl, darunter die Tastenbelegung für Bewegen, Heilen und Angreifen.



Spielfigur mit Lauf- und Angriffsbildern, Gegner, vier Nichtspielerfiguren und Tränke. Die Grafiken stammen vom Team, Animation, Kollision und Spiellogik dahinter von mir.

Mängelliste, von mir selbst geführt

Das Projekt ist von 2022 und war mein erstes größeres Stück Software. Diese Liste steht hier, weil sie zeigt, wie ich meinen eigenen Code heute lese.

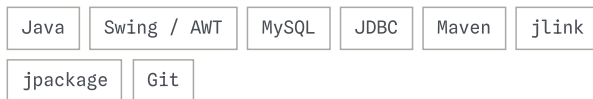
Zugangsdaten zur Datenbank stehen fest im Quelltext statt in einer Konfiguration

SQL wird per Zeichenkettenverkettung gebaut statt über Parameter

Die Oberflächenklasse ist mit 1.106 Zeilen deutlich zu groß geraten

Automatisierte Tests fehlen vollständig

WERKZEUGE



EINORDNUNG

Schulprojekt aus dem Jahr 2022. Der Quelltext stammt vollständig von mir, Grafiken und Klänge vom übrigen Team.

github.com/KarimBk7/Kaiju-Adventure

(<https://github.com/KarimBk7/Kaiju-Adventure>)

Technisches Lern-Repository

Über 17 Monate gewachsenes Repository mit eigenen Notizen, Terminal-Protokollen und lauffähigem Beispielcode zu 30 Themengebieten — von Python und Go über SQL und Linux-Administration bis zu Webentwicklung und Testen.

RAHMEN Programmierpraktikum, FU Berlin

BEGINN April 2025

MEINE ROLLE Alleinige Bearbeitung

STATUS 67 von 67 abgenommen

UMFANG 30 Themengebiete

AUFGABE

Das Programmierpraktikum der Freien Universität Berlin ersetzt Vorlesungen durch eigenständige Themenarbeit. Zu jedem Gebiet beantwortet man Verständnisfragen schriftlich, protokolliert die Arbeit an der Kommandozeile und lässt jede Aufgabe einzeln von einem Tutor abnehmen. Nicht bestandene Aufgaben gehen zur Überarbeitung zurück.

MEIN BEITRAG

Eigene Antworten in Markdown zu jedem Thema, in eigenen Worten statt übernommener Lösungen

Terminal-Protokolle mitgeschrieben, samt Fehlversuchen und Fehlermeldungen

Lauffähiger Beispielcode je Sprache, nicht nur Textausschnitte

Zeitaufwand je Aufgabe in der Commit-Nachricht erfasst

KENNWERTE

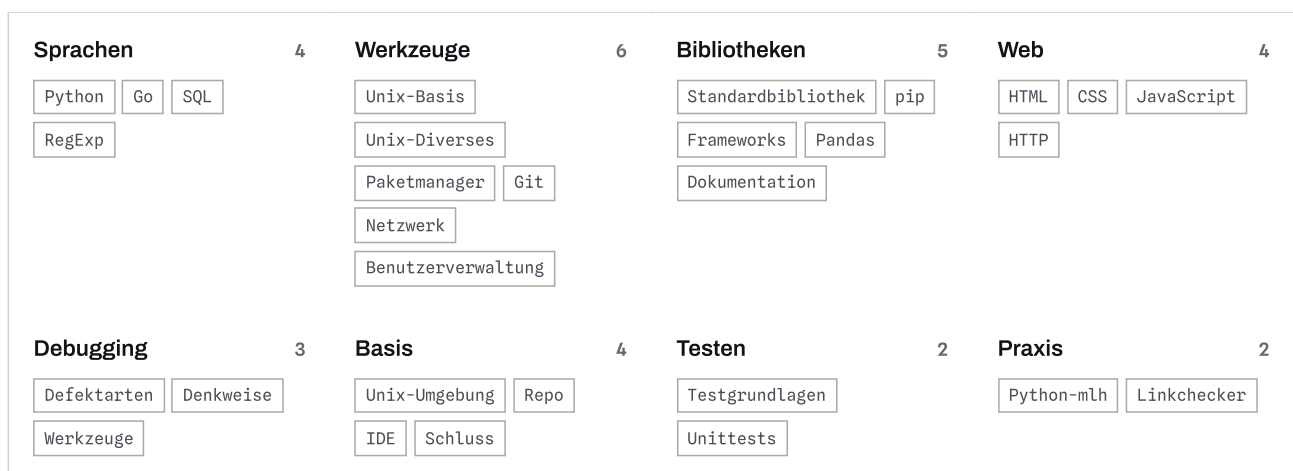
67 abgenommene Aufgaben
alle eingereichten

163 dokumentierte Stunden
aus 140 Commit-Nachrichten

356 Commits
April 2025 bis September 2026

156 eigene Dateien
Notizen, Protokolle, Code

THEMENLANDKARTE



WERKZEUGE

Python	Go	SQL	JavaScript	HTML / CSS	Linux
Bash	Git	pytest	Pandas	FastAPI	venv / pip

EINORDNUNG

Sämtliche Inhalte stammen von mir; die Fremdbeiträge im Repository beschränken sich auf das Abhaken des Abnahmestatus durch den Tutor. Ich nutze es als persönliches Nachschlagewerk weiter.

github.com/KarimBk7/Learning-repository

(<https://github.com/KarimBk7/Learning-repository>)

Wordle-Solver

Kleines Windows-Programm, das aus den Farbhinweisen einer Wordle-Runde alle noch möglichen Wörter herausfiltert — aus den echten Wortlisten des Spiels, in Python ohne eine einzige Abhängigkeit.

RAHMEN	Eigenarbeit
ZEITRAUM	Dez. 2025 – Sept. 2026
MEINE ROLLE	Gesamter Quelltext
UMFANG	367 Zeilen Python, drei Dateien
STAND	Release v1.0 auf GitHub

AUFGABE

Wordle gibt nach jedem Rateversuch drei Arten von Hinweisen: richtiger Buchstabe an richtiger Stelle, richtiger Buchstabe an falscher Stelle, Buchstabe gar nicht im Wort. Diese Hinweise im Kopf zu kombinieren ist mühsam und fehleranfällig. Das Programm nimmt sie als Eingabe und zeigt genau die Wörter, die noch übrig bleiben.

Erst erzeugen, dann filtern

Die erste Fassung baute aus den erlaubten Buchstaben alle denkbaren Zeichenketten und prüfte anschließend, welche davon in der Wortliste stehen. Das wächst exponentiell mit der Zahl der offenen Stellen. Die heutige Fassung dreht die Richtung um: Sie geht einmal über die Wortliste und prüft jedes Wort gegen die Bedingungen — feste Buchstaben, ausgeschlossene Positionen, erlaubte Buchstaben und die Häufigkeit je gelbem Buchstaben. Der Aufwand hängt damit nur noch an der Länge der Liste, nicht an der Zahl der Möglichkeiten.

MEIN BEITRAG

Filterlogik über beide Wortlisten, mit Häufigkeitsprüfung je Buchstabe

Oberfläche in Tkinter: fünf Spalten für die fünf Stellen, Tastatur zum Ausschließen grauer Buchstaben

Ergebnisse nach Anzahl verschiedener Buchstaben sortiert — das nächste Raten schließt so am meisten aus

Trennung in mögliche Lösungen und nur erlaubte Rateworte

Auslieferung als Windows-Ordner mit PyInstaller, startbar ohne Python

KENNWERTE

14.855 geprüfte Wörter
2.315 Lösungen, 12.540
Rateworte

2 ms je Suchlauf
Median aus 15 Läufen, Python
3.14

367 Zeilen Python
Oberfläche, Solver, Einstieg

0 Abhängigkeiten
nur die Standardbibliothek

	1st letter	2nd letter	3rd letter	4th letter	5th letter
Green (correct position)					
Can it appear again at another position?	☐ Yes ☐ No	☐ Yes ☐ No	☐ Yes ☐ No	☐ Yes ☐ No	☐ Yes ☐ No
Yellow (wrong position)					
How often in the word? 1 = exactly 1x 1+ = at least 1x	1+ ▾ 1+ ▾ 1+ ▾ 1+ ▾ 1+ ▾	1+ ▾ 1+ ▾ 1+ ▾ 1+ ▾ 1+ ▾	1+ ▾ 1+ ▾ 1+ ▾ 1+ ▾ 1+ ▾	1+ ▾ 1+ ▾ 1+ ▾ 1+ ▾ 1+ ▾	1+ ▾ 1+ ▾ 1+ ▾ 1+ ▾ 1+ ▾
Grey (not in word) click to exclude	Q W E R T Y U I O P A S D F G H J K L Z X C V B N M				
Find words					

Eingabefenster: je Spalte eine Stelle im Wort. Oben der grüne Buchstabe, darunter ob er noch einmal vorkommen darf, dann die gelben Buchstaben mit ihrer Häufigkeit, unten die Tastatur für die grauen.

Possible solutions (56)	Other allowed guesses (176)
black blade bland blank	beals bialy blads blady blaes
blast blaze chalk clang	blags blahs blain blanc blase
clank clash clasp dealt	blash blate blats blaud blawn
exalt flack flake flaky	blaws blays chals chawl clade
flank flash flask glade	clads claes clags clans claps
gland glaze place plaid	clapt clast clats claut clave
plain plait plane plank	clavi claws clays coals coaly
plant plate quail scald	deals dhals dials duals dwale
scale scalp scaly shale	elain eland elans feals flabs
shalt shawl slack slain	flags flaks flane flans flaps
slang slant slate slave	flats flawn flaws flawy flaxy
snail stale stalk whale	flays foals geals glace glads
avail clack class elate	glady glaik glans glazy goals
glass koala plaza slash	gyals heald heals kcals klang
	klaps neals odals olate opals
	ovals peals plack plage plaig

Close

Ergebnis in zwei Listen: links die Wörter, die noch die Lösung sein können, rechts die, die nur als Rateversuch erlaubt sind. Je Liste stehen Wörter mit den meisten verschiedenen Buchstaben oben.

Was fehlt

Ein Werkzeug für mich selbst, kein Produkt. Die Grenzen stehen hier, damit niemand sie erst suchen muss.

Keine automatisierten Tests; geprüft wurde von Hand gegen echte Spielrunden

Die Oberfläche liegt mit 305 Zeilen in einer einzigen Datei

Das Programm schlägt kein Wort vor, es filtert nur — eine Bewertung nach Informationsgewinn fehlt

Fertig gebaut nur für Windows, aus dem Quelltext läuft es überall

WERKZEUGE

[Python 3](#)[Tkinter](#)[PyInstaller](#)[Git](#)

AUSLIEFERUNG

Als ZIP mit Programmordner, entpacken und starten — Python wird nicht gebraucht. Bewusst als Ordner statt als einzelne EXE: eine selbstentpackende Datei stufen Virens Scanner und Browser regelmäßig als verdächtig ein.

Windows-Version herunterladen ZIP • 13 MB

([https://github.com/KarimBk7/Wordle-Solver](https://github.com/KarimBk7/Wordle-Solver/releases/latest/download/Wordle-Solver-windows.zip)

[Solver/releases/latest/download/Wordle-Solver-windows.zip](https://github.com/KarimBk7/Wordle-Solver/releases/latest/download/Wordle-Solver-windows.zip))

EINORDNUNG

Private Arbeit, der Quelltext stammt vollständig von mir. Die beiden Wortlisten sind die des Spiels und stammen aus öffentlichen Sammlungen; sie sind im Repository als Quelle genannt.

github.com/KarimBk7/Wordle-Solver

(<https://github.com/KarimBk7/Wordle-Solver>)